Homework 03

DataStream - Intervallo di confidenza per conteggio di elementi

In questo problema, studiamo un approccio per l'approssimazione della frequenza delle occorrenze di diversi elementi in uno stream di dati. Supponiamo di avere lo stream di dati $S = \{a_1, a_2, \ldots, a_t\}$ dove $a_i \in \{1, 2, \ldots, n\}$, ossia i nostri oggetti sono i numeri interi minori o uguali a n. Definiamo la frequenza $F[i]$ dell'oggetto i come

$$F[i] = \#\{a_j = i, j = 1, \ldots, t\}, \qquad 1 \leq i \leq n,$$

ossia $F[i]$ conta il numero di volte in cui i è apparso nello stream S. L'obbiettivo è avere sempre buone approssimazioni dei valori $F[i]$ $(1 \leq i \leq n)$.

Un modo semplice per farlo è semplicemente tenere i conteggi in memoria per ogni oggetto $1 \leq i \leq n$ separatamente. Tuttavia, ciò richiederà $O(n)$ spazio e in molte applicazioni questo può essere proibitivo (per esempio, per il conteggio dei clic degli utenti sugli annunci pubblicitari online), specialmente nei contesti in cui $n \gg t$ (sparsità).

Esercizio È possibile approssimare approssimare $F[i]$ attraverso la stima $\hat{F}[i]$ data nell'Algoritmo 1, per cui sarà necessario uno spazio di solo $O(\frac{1}{\epsilon}\log\frac{1}{\delta})$ e per il quale vale che, per ogni $i = 1, \ldots, n$,

$$P(F[i] \leq \hat{F}[i] \leq F[i] + \epsilon t) \geq 1 - \delta.$$

Algoritmo 1: Stima del numero di oggetti distinti $F[i]$ in uno stream di dati S

Algoritmo 1

> INPUT: a stream S, two parameters $\delta > 0, \epsilon > 0$;
> IN MEMORY: an integer matrix H of counters
> $H_{k,j}, k \in \{0, \ldots, K-1\}, j \in \{0, \ldots, J-1\}$;
> QUERYING OUTPUT: for any $1 \leq i \leq n$, estimation of $F[i]$ with $\hat{F}[i]$ given below;
>
> // Initialization of hash functions and memory space
> $K \longleftarrow \lceil \log \frac{1}{\delta} \rceil$;

$J \longleftarrow \lceil \frac{e}{\epsilon} \rceil$;

// definition of the K hash functions
FOR $k = 0, \ldots, K$:
$h_k : \mathbb{Z}_+ \to \{0, \ldots, J-1\}$;
ENDFOR

// initialization of memory space: set to 0 the matrix of counters
$H_{k,j} = 0$, $k = 0, \ldots, K-1$, $j = 0, \ldots, J-1$

// Streaming Algorithm
WHILE $\ldots, a_l, \ldots$ in stream S:
FOR $k \in \{0, \ldots, K-1\}$:
$H_{k,h_k(a_l)} \longleftarrow H_{k,h_k(a_l)} + 1$;
ENDFOR
ENDWHILE

// Querying Algorithm
INPUT: an integer i;
RETURN $\hat{F}[i] = \min_{k=0,\ldots,K-1} H_{k,h_k(i)}$.

Idea alla base dell'algoritmo

Prendiamo una sola funzione hash $h : \mathbb{N} \to \{1, \ldots, J\}$, e teniamo J contatori H_j. Quando arriva l'oggetto i, aumentiamo il contatore $H_{h(i)}$. A questo punto, vari oggetti potranno essere mappati nello stesso valore da h, e quindi ci sarà un problema nella stima finale quando un oggetto poco frequente viene mappato insieme a uno molto frequente.
Lampadina: se però abbiamo K funzioni hash indipendenti, la probabilità che un oggetto sia *sempre* mappato con oggetti molto frequenti diminuisce\ldots

Descrizione algoritmo

In questo esercizio proveremo l'Algoritmo 1 su centinaia di milioni di dati.

Inizializzazione. L'algoritmo dipende da due parametri $\delta > 0, \epsilon > 0$. In funzione di questi, definiamo due numeri interi $K = \lceil \log \frac{1}{\delta} \rceil$ e $J = \lceil \frac{e}{\epsilon} \rceil$. Vengono usate

- K funzioni hash $g_0, \ldots, g_{K-1}$ con valori ammissibili $\{0, 1, \ldots, N\}$, dove N è molto maggiore di J, cosicché potremo sempre supporre che

$$h_k(i) = g_k(i) \bmod J, \qquad k = 0, \ldots, K-1,$$

  siano k *funzioni hash indipendenti* con valori in $\{0, 1, \ldots, J-1\}$;
- una matrice H di $K \times J$ contatori, inizialmente messi tutti a 0.

Fase stream. Ogniqualvolta arriva un nuovo oggetto i, vengono valutate le k funzioni hash per i e vengono aumentati di 1 i K contatori $H_{k,h_k(i)}$, $k = 0, \ldots, K-1$.

Fase di interrogazione. Dato i, si stima $\hat{F}[i] = \min_k H_{k,h_k(i)}$.

Datastream in Python

Attenzione: svilupperemo l'algoritmo in Python, non avendo tra l'altro a disposizione abbastanza risorse per garantire l'analisi di tutta la mole di quei dati (con repliche e fault tollerance come fa Spark) in tempi ragionevoli.

Funzioni hash indipendenti

Eseguire l'algoritmo sui dati presentati con $\delta = e^{-5}$, $\epsilon = e \cdot 10^{-4}$. Con questi parametri, occorrono $K = \lceil \log \frac{1}{\delta} \rceil = 5$ funzioni hash. Le seguenti funzioni hash garantiscono una **buona casualità** unita alla velocità di esecuzione e saranno quindi predefinite nel codice:

$$
\begin{aligned}
g_0(i) &= 3i + 1561 \bmod 123457; \\
g_1(i) &= 17i + 277 \bmod 123457; \\
g_2(i) &= 38i + 394 \bmod 123457; \\
g_3(i) &= 61i + 13 \bmod 123457; \\
g_4(i) &= 78i + 246 \bmod 123457.
\end{aligned}
$$

Learning by examples

Confronteremo le seguenti tecniche di *reinforced learning*

1. Epsilon-Greedy;
2. UCB1 (Upper confidence sampling);
3. Thompson sampling, eventualmente modificato opportunamente.

Per fare ciò, avremo a disposizione la classe `BanditGame` discussa a lezione.

Obiettivo è confrontare le strategie sopra avendo a disposizione $T = 10\,000$ in diversi contesti con a disposizione 4 arms possibili.